

Part 2. Real / Fake News Classification (Section 5 - Section 7)

You've just joined UMD News as a data scientist, and the Fact-Checking Department is begging for help 🙏🙏🙏.

They've been working 24/7 trying to spot fake news, and let's just say... morale is low and coffee supplies are dangerously high ☕⚠️.

They need you to **build a fake news classifier** to help catch false stories automatically—so they can finally take a break (or at least a nap) 🛌😴.

They've sent you a dataset to get started.

Section 5: Download, Assess, and Preprocess Data (20 points)

5.1. Download the data (0 pts)

Put the two csv files (real.csv and fake.csv) into the same directory of this file. Here is a piece of code to check whether you did correctly.

```
In [1]: import os
import pandas as pd

files = os.listdir('.')
print("real.csv is in the folder:", "real.csv" in files)
print("fake.csv is in the folder:", "fake.csv" in files)
```

```
real.csv is in the folder: True
fake.csv is in the folder: True
```

5.2. Load the data and have a first glance (5 pts)

Look at the dataset. In the box below, write a paragraph describing the dataset.

```
In [2]: # You may write any code you need here.
real_df = pd.read_csv('real.csv')
fake_df = pd.read_csv('fake.csv')
print("Real dataset shape:", real_df.shape)
print("Fake dataset shape:", fake_df.shape)
print("First 3 rows of the real dataset:")
print(real_df.head(3))
print("\n\nFirst 3 rows of the fake dataset:")
print(fake_df.head(3))
real_df.dtypes
```

Real dataset shape: (5000, 4)

Fake dataset shape: (5000, 4)

First 3 rows of the real dataset:

	title \	text	subject \	date
0	Thousands march in Helsinki in far-right, anti...		worldnews	December 6, 2017
1	Marseille attacker probably radicalized by bro...		worldnews	October 9, 2017
2	U.S. farmers slam Trump's Cuba clampdown, pres...		politicsNews	June 16, 2017

First 3 rows of the fake dataset:

	title \	text	subject	date
0	EP #15: Patrick Henningsen LIVE - 'Crisis of A...			
1	Desperate For Members, White Supremacists Beg...			
2	Black Caucus Demands Action: Calls On Congres...			

	text	subject	date
0	Join Patrick every Wednesday at Independent T...	US_News	March 2, 2017
1	By now, everyone is aware of how Donald Trump ...	News	August 10, 2017
2	Following the horrific tragedy in Dallas, Texa...	News	July 8, 2016

Out[2]:

0	
title	object
text	object
subject	object
date	object

dtype: object

Paragraph with answer here. One point each for:

- writing a paragraph
- the size of the dataset
- what each file represents
- what each column represents
- the datatype of each column

Each of the two CSV files has a shape of 5000 rows and 4 columns, which combines for 10000 rows and 4 columns due to the columns all being the same. Each column represents critical information of a specific news report, with the 4 columns being the title of the

headline, the article itself in the news report, the subject of said report, and the date of the article's publication. The two files represent real news reports and fake news reports. All of the columns' datatypes for both datasets are of type Object, in which they are simply Strings.

5.3. What do you think? (4 pts)

You get home from work and your friend asks you about your job. What do you think about training a classifier on this dataset?

Paragraph with answer here. We expect them to write how this is easy (all the true examples have cities associated with them) or hard (3 points), and the context of how this classifier may be used (2 points).

I think this will not be that difficult of a task. The reason why I think this is pretty doable is because I have detected a pattern between the text of the real news and fake news, in which the real news' texts almost always start with the name of a city or the name of the news source like Reuters. I think we can use this to differentiate between real and fake news.

5.4. Check for empty entries (3 pts)

Your colleague told you: "Hey, some entries may be empty." Is that true? Count the number of entries per column that don't have information in them.

```
In [3]: def count_empty_entries(df):
        for column in df.columns:
            ### TODO Add Your Code Here: Count the number of empty entries in each column
            empty_count = df[column].isin(['', 'N/A', ' ']).sum()
            print(f"Column '{column}' has {empty_count} empty entries.")

        print("Counting empty entries in the real dataset:")
        count_empty_entries(real_df)
        print("\nCounting empty entries in the real dataset:")
        count_empty_entries(fake_df)
```

```
Counting empty entries in the real dataset:
Column 'title' has 6 empty entries.
Column 'text' has 6 empty entries.
Column 'subject' has 4 empty entries.
Column 'date' has 5 empty entries.
```

```
Counting empty entries in the real dataset:
Column 'title' has 5 empty entries.
Column 'text' has 142 empty entries.
Column 'subject' has 5 empty entries.
Column 'date' has 10 empty entries.
```

5.5. Delete some columns and rows (1 + 2 pts)

After discussing with your team leader, you both believe **subject** and **date** are not **useful for your analysis**, so you decide to **drop them** from both datasets.

In addition, you will **remove any rows that have empty entries in the title or text column** from both datasets.

```
In [4]: # 1 point
def remove_subject_and_date(df):
    # TODO Add your code here: drop the 'subject' and 'date' columns from the dataframe
    df = df.drop(columns=['subject', 'date'])
    return df

real_df = remove_subject_and_date(real_df)
fake_df = remove_subject_and_date(fake_df)
print("real_df's current columns:", real_df.columns)
print("fake_df's current columns:", fake_df.columns)
```

```
real_df's current columns: Index(['title', 'text'], dtype='object')
fake_df's current columns: Index(['title', 'text'], dtype='object')
```

```
In [5]: # 2 points
def remove_rows_with_empty_entries(df):
    # TODO Add your code here: remove rows with any empty entries in the dataframe
    df = df[~(df == ' ').any(axis=1)]
    return df

real_df = remove_rows_with_empty_entries(real_df)
fake_df = remove_rows_with_empty_entries(fake_df)
print("After removing rows with empty entries:")
print("Real dataset shape after cleaning:", real_df.shape)
print("Fake dataset shape after cleaning:", fake_df.shape)
```

After removing rows with empty entries:

Real dataset shape after cleaning: (4988, 2)

Fake dataset shape after cleaning: (4853, 2)

5.6. Remove special characters (5 pts)

You may notice there are many special characters in the text data, such as punctuation marks, numbers, and other non-alphabetic characters.

Please **remove all these special characters** from the text data in both datasets. **Only keep alphabetic characters and spaces.**

In addition, **make all the characters lower-case.**

```
In [6]: def remove_special_characters(df):
    # TODO Add your code here: remove special characters from both columns.
    df['title'] = df['title'].str.replace('[^a-zA-Z\s]', '', regex=True).str.lower()
    df['text'] = df['text'].str.replace('[^a-zA-Z\s]', '', regex=True).str.lower()

    return df

real_df = remove_special_characters(real_df)
fake_df = remove_special_characters(fake_df)
print("After removing special characters:")
```

```
print("Real dataset shape after cleaning:", real_df.head(3))
print("\n\nFake dataset shape after cleaning:", fake_df.head(3))
```

```
<>:3: SyntaxWarning: invalid escape sequence '\s'
<>:4: SyntaxWarning: invalid escape sequence '\s'
<>:3: SyntaxWarning: invalid escape sequence '\s'
<>:4: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_1368/2192013692.py:3: SyntaxWarning: invalid escape sequence '\s'
  df['title'] = df['title'].str.replace('[^a-zA-Z\s]', '', regex=True).str.lower()
/tmp/ipykernel_1368/2192013692.py:4: SyntaxWarning: invalid escape sequence '\s'
  df['text'] = df['text'].str.replace('[^a-zA-Z\s]', '', regex=True).str.lower()
```

After removing special characters:

Real dataset shape after cleaning:

ti

title \

```
0 thousands march in helsinki in farright antifa...
1 marseille attacker probably radicalized by bro...
2 us farmers slam trumps cuba clampdown press fo...
```

text

```
0 helsinki reuters supporters of the far right ...
1 rome reuters the brother of the man who kille...
2 chicago reuters us farm groups criticized pre...
```

Fake dataset shape after cleaning:

ti

title \

```
0 ep patrick henningsen live crisis of america...
1 desperate for members white supremacists beg ...
2 black caucus demands action calls on congress...
```

text

```
0 join patrick every wednesday at independent t...
1 by now everyone is aware of how donald trump s...
2 following the horrific tragedy in dallas texas...
```

Section 6. Create the training and testing test (6 pts)

Now we have finished pre-processing the data. We are now constructing a training set and a testing set to build up and evaluate our classifier.

We will follow the steps:

1. (6.1.) Assign labels to each row.
2. (6.2.) Concatenate the two tables `real_df` and `fake_df` into one data frame `union_df`.
3. (6.3.) Split the `union_df` into `train_df` and `test_df`, where the training/testing set contains 80%/20% of the data.

6.1. Assign labels to each row (2 pts)

Add a `label` column to each of the data frame. For `real_df`, the `label` column is all 1. For `fake_df`, the `label` column is all 0.

```
In [7]: def add_label_column(df, value):
# TODO Add your code here: add a new column 'label' to the dataframe with the s
df['label'] = value

return df

real_df = add_label_column(real_df, 1) # Label for real emails
fake_df = add_label_column(fake_df, 0) # Label for fake emails
print("After adding label column:")
print("Real dataset shape with labels:", real_df.head(3))
print("\n\nFake dataset shape with labels:", fake_df.head(3))
```

After adding label column:

Real dataset shape with labels:

title

```
\
0 thousands march in helsinki in farright antifa...
1 marseille attacker probably radicalized by bro...
2 us farmers slam trumps cuba clampdown press fo...
```

	text	label
0	helsinki reuters supporters of the far right ...	1
1	rome reuters the brother of the man who kille...	1
2	chicago reuters us farm groups criticized pre...	1

Fake dataset shape with labels:

title

```
\
0 ep patrick henningsen live crisis of america...
1 desperate for members white supremacists beg ...
2 black caucus demands action calls on congress...
```

	text	label
0	join patrick every wednesday at independent t...	0
1	by now everyone is aware of how donald trump s...	0
2	following the horrific tragedy in dallas texas...	0

6.2. Concatenate the two data frames into one (2 pts)

```
In [8]: def concatenate_dataframes(df1, df2):
# TODO Add your code here: concatenate the two dataframes
combined_df = pd.concat([df1, df2], ignore_index=True)

return combined_df

union_df = concatenate_dataframes(real_df, fake_df)
print("Combined dataset shape:", union_df.shape)
print("First 3 rows of the combined dataset:")
print(union_df.head(3))
print("\n\nLast 3 rows of the combined dataset:")
print(union_df.tail(3))
```

Combined dataset shape: (9841, 3)

First 3 rows of the combined dataset:

	title \	text	label
0	thousands march in helsinki in farright antifa...	helsinki reuters supporters of the far right ...	1
1	marseille attacker probably radicalized by bro...	rome reuters the brother of the man who kille...	1
2	us farmers slam trumps cuba clampdown press fo...	chicago reuters us farm groups criticized pre...	1

Last 3 rows of the combined dataset:

	title \	text	label
9838	cnns jim acosta schooled on the meaning of the...	we wish president trump could clone senior adv...	0
9839	donald trumps first campaign tv ad is here an...	while an armed militia group of domestic terro...	0
9840	one brutal image perfectly captures the truth ...	a spokesperson for the clinton foundation clai...	0

6.3. Split the dataset into training and testing sets (2 pts)

There are many ways to implement this.

The printed results don't need to be exactly the same as the expected output, but they should be close.

```
In [9]: def split_dataset(df, train_size=0.8):
# TODO Add your code here: split the dataset into training and testing sets
q = int(len(df) * train_size)
train_df = df[:q]
test_df = df[q:]

return train_df, test_df

train_df, test_df = split_dataset(union_df, train_size=0.8)
print("Training dataset shape:", train_df.shape)
print("Testing dataset shape:", test_df.shape)
print("Proportion of training set:", len(train_df) / len(union_df))
print("Proportion of testing set:", len(test_df) / len(union_df))
print("#Positive samples in training set:", len(train_df[train_df['label'] == 1]))
print("#Negative samples in training set:", len(train_df[train_df['label'] == 0]))
print("#Positive samples in testing set:", len(test_df[test_df['label'] == 1]))
print("#Negative samples in testing set:", len(test_df[test_df['label'] == 0]))
```

```

Training dataset shape: (7872, 3)
Testing dataset shape: (1969, 3)
Proportion of training set: 0.7999187074484301
Proportion of testing set: 0.20008129255156995
#Positive samples in training set: 4988
#Negative samples in training set: 2884
#Positive samples in testing set: 0
#Negative samples in testing set: 1969

```

Section 7: Build a classifier and evaluate its performance (29 pts)

Now we have our training set and testing set ready. We are going to train classifiers and evaluate its performance.

After discussing with your team leader,

- You are going to first encode the news, which means convert each piece of news into a fixed-length vector. And then you will train a classifier.
- You are going to try two encoders, `TfidfVectorizer` and `SentenceTransformer`. You will use `Linear SVM` on the TF-IDF vectors and `Logistic Regression` on the `SentenceTransformer` vectors.
- For this assignment, use **only the text column** (article body) as the document to classify; do not use the headline (`title`) as input.

Fill in the **classification accuracy on the test set** into the table at the end.

	<code>TfidfVectorizer</code> + <code>Linear SVM</code>	<code>SentenceTransformer</code> + <code>Logistic Regression</code>
<code>text</code> only		

7.1: `TfidfVectorizer` - `Linear SVM` (13 pts)

7.1.1. `TfidfVectorizer` (5 pts)

Currently, our input is a string. Machine learning models like `Linear SVM` cannot receive strings as inputs. Therefore, our first step is to convert the input strings into feature vectors. We will use `TfidfVectorizer` from `sklearn.feature_extraction.text` to achieve this. You are recommended to read the documentation to facilitate your coding.

You would do the following:

1. Fit the `TfidfVectorizer` using the **training text** column.
2. Use the fitted vectorizer to transform the **test text** column.

```

In [10]: def extract_feature_vectors(train_df_input_column, test_df_input_column):
          # TODO Add your code here: first fit the TfidfVectorizer on the training set, t
          from sklearn.feature_extraction.text import TfidfVectorizer

```



```

vectorizer = TfidfVectorizer(max_features=1000)
train_feature_vectors = vectorizer.fit_transform(train_df_input_column)
test_feature_vectors = vectorizer.transform(test_df_input_column)

return train_feature_vectors, test_feature_vectors

train_tfidf_vectors, test_tfidf_vectors = extract_feature_vectors(train_df["text"],
print("TF-IDF feature vectors (train):", train_tfidf_vectors.shape)
print("TF-IDF feature vectors (test): ", test_tfidf_vectors.shape)

```

TF-IDF feature vectors (train): (7872, 1000)

TF-IDF feature vectors (test): (1969, 1000)

7.1.2. Linear SVM (8 pts)

Train a **Linear SVM model** on the training set and evaluate its performance on the testing set. Feel free to set your own hyper-parameters.

```

In [11]: def trainSVM(train_feature_vectors, train_labels, test_feature_vectors, test_labels
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report

# Create and train the model
model = LinearSVC()
model.fit(train_feature_vectors, train_labels)
# Make predictions on the test set
predictions = model.predict(test_feature_vectors)
# Evaluate the model
accuracy = accuracy_score(test_labels, predictions)
report = classification_report(test_labels, predictions)
print("Accuracy of Linear SVM Model:", accuracy)
print("Classification Report:\n", report)
return model, accuracy, report

# Prepare labels for training and testing
train_labels = train_df['label'].values
test_labels = test_df['label'].values
# Linear SVM on TF-IDF features (text column only)
model_tfidf_svm, accuracy_tfidf_svm, report_tfidf_svm = trainSVM(
    train_tfidf_vectors, train_labels,
    test_tfidf_vectors, test_labels,
)
print("Accuracy of Linear SVM (TF-IDF, text only):", accuracy_tfidf_svm)

```

Accuracy of Linear SVM Model: 0.9801929913661758

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.98	0.99	1969
1	0.00	0.00	0.00	0
accuracy			0.98	1969
macro avg	0.50	0.49	0.49	1969
weighted avg	1.00	0.98	0.99	1969

Accuracy of Linear SVM (TF-IDF, text only): 0.9801929913661758

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Und
efinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no tr
ue samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Und
efinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no tr
ue samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Und
efinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no tr
ue samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

7.2. SentenceTransformer - Logistic Regression (11 pts)

You will use a python package named `sentence_transformers` to convert a sentence into a 384-dimensional vector. You are encouraged to read through the documentation:

<https://sbert.net/>, which describes its simple interface.

7.2.1. Install Sentence Transformer (3 pts)

First you will install the sentence-transformers package using pip. `pip install -U sentence-transformers` After installation, you can test-run the following codes from <https://sbert.net/> to see whether it is successfully installed.

```

In [12]: import torch
        from sentence_transformers import SentenceTransformer

def _demo_device():
    if torch.cuda.is_available():
        return "cuda"
    if getattr(torch.backends, "mps", None) is not None and torch.backends.mps.is_a
        return "mps"
    return "cpu"

# 1. Load a pretrained Sentence Transformer model (same name as TASK 3.2.2 for a si
model = SentenceTransformer("paraphrase-MiniLM-L3-v2", device=_demo_device())
print(f"Using device: {model.device}")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 384]

# 3. Calculate the embedding similarities
similarities = model.similarity(embeddings, embeddings)

```

```
print(similarities)
# tensor([[1.0000, 0.5960, 0.0138],
#         [0.5960, 1.0000, 0.0906],
#         [0.0138, 0.0906, 1.0000]])
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated request
s to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downl
oads.
```

```
modules.json: 100% ██████████ 229/229 [00:00<00:00, 19.2kB/s]
config_sentence_transformers.json: 100% ██████████ 122/122 [00:00<00:00, 11.4kB/s]
README.md: 100% ██████████ 3.83k/3.83k [00:00<00:00, 354kB/s]
sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 5.99kB/s]
config.json: 100% ██████████ 629/629 [00:00<00:00, 51.2kB/s]
model.safetensors: reconstructing file: 100% ██████████ 69.6MB / 69.6MB, 6.66MB/s
model.safetensors: downloading bytes: ██████████ 65.5MB, 6.17MB/s
Loading weights: 100% ██████████ 55/55 [00:00<00:00, 2917.76it/s]
tokenizer_config.json: 100% ██████████ 314/314 [00:00<00:00, 5.18kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 11.3MB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 24.7MB/s]
special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 12.8kB/s]
config.json: 100% ██████████ 190/190 [00:00<00:00, 22.0kB/s]
Using device: cuda:0
(3, 384)
tensor([[1.0000, 0.5960, 0.0138],
        [0.5960, 1.0000, 0.0906],
        [0.0138, 0.0906, 1.0000]])
```

7.2.2. Transform the text column into vectors using SentenceTransformer (5 pts)

Warning: This may take longer than expected to run. The code cell saves embeddings to `hw5_st_embeddings_text.npz` after the first full run; later runs load from disk (delete the file to recompute). On **Colab**, use **Runtime** → **Change runtime type** → **GPU** for a large speedup; **Apple Silicon** may use **MPS** automatically(ensure you have torch GPU installed instead of CPU).

Warning: In our test, running this cell on **CPU** took **over 312 seconds** with 2 CPU core, **38 sec** on 48 CPU cores. On **GPU**, the same operation took **22 seconds**.

For Google Colab users (including the free tier): you can enable GPU acceleration by clicking **Runtime** → **Change runtime type** → **T4 GPU** → **Save**.

In [13]: `print(os.cpu_count())`

2

```
In [14]: import os
import time

import numpy as np
import torch
from sentence_transformers import SentenceTransformer

# We suggest `paraphrase-MiniLM-L3-v2` model
ST_MODEL_NAME = "paraphrase-MiniLM-L3-v2"

# Disk cache (text column only).
_ST_EMBEDDINGS_CACHE = "hw5_st_embeddings_text.npz"
_st_model = None

def _st_device():
    if torch.cuda.is_available():
        return "cuda"
    if getattr(torch.backends, "mps", None) is not None and torch.backends.mps.is_a
        return "mps"
    return "cpu"

def _st_batch_size(device: str) -> int:
    if device == "cuda":
        return 256
    if device == "mps":
        return 128
    return min(96, max(32, (os.cpu_count() or 4) * 4))

def get_sentence_model():
    """Load once. Uses GPU (CUDA), Apple MPS, or CPU."""
    global _st_model
    if _st_model is None:
        try:
            demo = model # from TASK 3.2.1 cell if you ran it first
            if demo is not None:
                _st_model = demo
                return _st_model
        except NameError:
            pass

    device = _st_device()
    if device == "cpu":
        torch.set_num_threads(min(8, os.cpu_count() or 1))

    # TODO: Load the pretrained model into _st_model (use ST_MODEL_NAME and dev
```

```

        _st_model = SentenceTransformer(ST_MODEL_NAME, device=device)
    return _st_model

def transform_text_column(train_col, test_col, model=None):
    model = model or get_sentence_model()
    dev_t = getattr(model.device, "type", _st_device())
    bs = _st_batch_size(dev_t if dev_t in ("cuda", "mps", "cpu") else _st_device())
    kwargs = dict(batch_size=bs, show_progress_bar=False)
    # TODO: encode train_col and test_col with model.encode(...)
    train_vecs = model.encode(train_col.tolist(), **kwargs)
    test_vecs = model.encode(test_col.tolist(), **kwargs)
    return train_vecs, test_vecs

timer1 = time.time()

if os.path.isfile(_ST_EMBEDDINGS_CACHE):
    z = np.load(_ST_EMBEDDINGS_CACHE)
    train_st_vectors = z["train"]
    test_st_vectors = z["test"]
    print("Loaded cached SentenceTransformer embeddings from", _ST_EMBEDDINGS_CACHE)
else:
    train_st_vectors, test_st_vectors = transform_text_column(train_df["text"], test_df["text"], model)
    print("SentenceTransformer vectors (train):", train_st_vectors.shape)
    print("SentenceTransformer vectors (test): ", test_st_vectors.shape)
    np.savez(_ST_EMBEDDINGS_CACHE, train=train_st_vectors, test=test_st_vectors)
    print("Saved embeddings to", _ST_EMBEDDINGS_CACHE, "(delete this file to recompute)")

timer2 = time.time()
print(f"Elapsed time: {timer2 - timer1} seconds")

```

```

SentenceTransformer vectors (train): (7872, 384)
SentenceTransformer vectors (test): (1969, 384)
Saved embeddings to hw5_st_embeddings_text.npz (delete this file to recompute)
Elapsed time: 18.53656578063965 seconds

```

Hey look! It's linear! What does this mean? By casting into higher dimensional spaces through a **kernel function**, we can linearly separate our data.

Of course we chose a very simple kernel.

7.2.3. Logistic Regression (You can reuse majority of the code from TASK 3.1.2, see how robust the package is designed) (3 pts)

Train a **Logistic Regression model** on the training set and evaluate its performance on the testing set. Feel free to set your own hyper-parameters.

Consider using `accuracy_score` and `classification_report` from `sklearn.metrics` to evaluate and present your results.

You don't need to match the exact values from `expected_output.ipynb`, but you shall obtain a similar format of outputs and reasonable values (e.g. accuracy not lower than 0.85).

```
In [15]: def trainLogisticRegression(train_feature_vectors, train_labels, test_feature_vectors, test_labels):
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import accuracy_score, classification_report

    # Create and train the model
    model = LogisticRegression()
    model.fit(train_feature_vectors, train_labels)
    # Make predictions on the test set
    predictions = model.predict(test_feature_vectors)
    accuracy = accuracy_score(test_labels, predictions)
    report = classification_report(test_labels, predictions)
    # Evaluate the model
    print("Accuracy of Logistic Regression Model:", accuracy)
    print("Classification Report:\n", report)
    return model, accuracy, report

# Prepare labels for training and testing
train_labels = train_df['label'].values
test_labels = test_df['label'].values
# Logistic Regression on SentenceTransformer features (text column only)
model_st_lr, accuracy_st_lr, report_st_lr = trainLogisticRegression(
    train_st_vectors, train_labels,
    test_st_vectors, test_labels,
)
print("Accuracy of Logistic Regression (SentenceTransformer, text only):", accuracy_st_lr)
```

Accuracy of Logistic Regression Model: 0.9558151345860844

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.96	0.98	1969
1	0.00	0.00	0.00	0
accuracy			0.96	1969
macro avg	0.50	0.48	0.49	1969
weighted avg	1.00	0.96	0.98	1969

Accuracy of Logistic Regression (SentenceTransformer, text only): 0.9558151345860844

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

TASK 7.3. Report to your team leader (5 pts)

Finally, fill in the accuracy into the table according to your experiments.

TfIdfVectorizer + Linear SVM SentenceTransformer + Logistic Regression

text	only	0.98
------	------	------

		0.96
--	--	------

Send your team leader a message : "Hi manager, I recommend using [???] as the encoder on the **text** column. This gives an accuracy of [???]."

Two minutes later, your team leader replies:

"Wonderful job!!! I will promote you as the senior data scientist next year!"

"Hi manager, I recommend using TfIdfVectorizer + Linear SVM as the encoder on the **text** column. This gives an accuracy of **0.98**."